

Rinata AI Agent Suite

Automated intelligence for a Minneapolis Italian restaurant

BUILD 1 COMPLETE

BUILD 2 COMPLETE

BUILD 6 COMPLETE

BUILDS 3 & 4 IN PROGRESS

RESTAURANT	REPORT DATE	BUILDS COMPLETE	STACK
Rinata — Minneapolis, MN	May 2026	3 of 8	Python · Claude API · Gmail API · n8n

Executive Summary

The Rinata AI Agent Suite is a collection of purpose-built Python agents that automate the highest-friction, most repetitive operational tasks at Rinata restaurant. Each build is a standalone pipeline that ingests real restaurant data, processes it through Claude (Anthropic's AI), and outputs a ready-to-use deliverable — an Excel workbook, a response draft, or a priority queue — that the manager can act on in minutes rather than hours.

The suite runs on a **thin harness / fat skills** architecture: 85% deterministic Python code handles data ingestion, cleaning, and formatting; Claude handles only the judgment calls (categorization, tone, summarization). This makes the system fast, auditable, and cheap to operate.

Three builds are complete as of this report. Together they cover guest review intelligence, owner response drafting, and supplier email triage — eliminating an estimated 6–10 hours of weekly manual work.

Complete Build Status

BUILD 1

Feedback & Reviews Intelligence System

✓ COMPLETE

Scrapes Google Reviews, cleans and categorizes every review using Claude, and produces a multi-sheet Excel analysis workbook with sentiment trends, top themes, and staff mentions.

BUILD 2

Review Response Agent

✓ COMPLETE

Extends Build 1. Generates on-brand draft responses for every unanswered review. Urgent reviews (1–2 star, health keywords) are routed to a priority queue. Hardened post-sign-off with a 3★-and-below filter, idempotent reruns, and a Docker-based n8n approval workflow.

BUILD 6

Supplier Email Triage Agent

✓ BUILT — SETUP REQUIRED

Connects to the restaurant's Google Workspace Gmail, fetches supplier emails, categorizes each one (invoice, delivery, price change, dispute), flags urgent items, and outputs a colour-coded Excel triage workbook.

BUILD 3

Weekly P&L Drafting Agent

BLOCKED

Ingests weekly financials and generates a formatted P&L with variance analysis and management narrative.

⚠ Waiting: QuickBooks login + ownership sign-off on P&L targets

BUILD 4

Vendor Invoice Reconciliation Agent

BLOCKED

Parses vendor invoice PDFs, matches line items against a master price list, and flags discrepancies.

⚠ Waiting: Vendor invoice PDFs + master price list

BUILDS 5, 7, 8

Menu / Scheduling / Reservations

PLANNED

Menu profitability analysis, scheduling assistant (most complex), and reservation pattern analysis. Scoped but not yet built.

What Was Built

Build 6 connects to Rinata's Google Workspace Gmail account, pulls emails from supplier and vendor contacts, and uses Claude to categorize and prioritize each one. The output is a four-sheet Excel workbook the manager opens every morning to see exactly what needs attention — and in what order.

Pipeline Architecture



Email Categories

Category	Description	Urgency	Example
price_change	Vendor announcing price increases or decreases on any item	Always High	"Chicken breast pricing update effective June 1"
dispute	Wrong items, shortages, damaged goods, billing errors	Always High	"Short shipment on your June 3 order — 2 cases missing"
invoice	Vendor billing for goods delivered; statements; remittance	Medium (High if past-due)	"Invoice #4821 — \$1,240.00 due June 15"
delivery	Shipping notices, ETAs, delivery confirmations, delays	Medium (High if delay)	"Your Sysco order ships Tuesday AM, ETA 7–9am"
order_confirm	Vendor confirming a purchase order placed by the restaurant	Low	"Order #2091 confirmed — 3 cases tomatoes, 2 cases pasta"
promo	Vendor marketing, sales promotions, product announcements	Low	"Summer specials — 10% off seasonal produce this week"
general	Anything else: introductions, relationship emails, miscellaneous	Low	"Introducing your new account rep, Maria Johnson"

Output Workbook — 4 Sheets

Sheet 1 — All Emails

Full list of all supplier emails, colour-coded by urgency (red = high, amber = medium, green = low). Sortable and filterable. Every email includes: date, supplier name, subject, category, urgency, one-sentence summary, and a plain-English action item.

Sheet 2 — Urgent Queue

High-urgency emails only, sorted with price changes and disputes at the top. This is the first thing to check each morning. Price changes need a decision; disputes need a call.

Sheet 3 — Invoices

All invoice emails in one place for accounts payable. Each row includes the extracted amount, due date, and invoice number where available.

Sheet 4 — Price Changes

Price change notices isolated for review. These are the emails that compound silently — a 5% increase on chicken breast adds up to thousands of dollars a year if missed.

Files Created

Rinata/ ├── email_triage/ | | ├── suppliers.json ← Known supplier domains + keyword search terms (edit this) | | |
gmail_auth.py ← One-time OAuth setup script | | ├── fetch_emails.py ← Pulls emails from Gmail API | | |
categorize_emails.py ← Claude Haiku categorization engine | | ├── build_triage_sheet.py ← Builds the Excel workbook | | |
└── run_email_triage.py ← Full pipeline orchestrator | | ├── prompts/ | | └── email_triage_v1.txt ← Categorization system
prompt └── output/ | ├── raw_emails.json ← Generated: raw fetched email data | ├── email_categories.csv ← Generated:
categorized results | ├── urgent_emails.csv ← Generated: high-urgency subset | └── Rinata_Email_Triage.xlsx ← Generated:
final deliverable

Getting Build 6 Running

Prerequisites

You need the restaurant's Google Workspace login credentials and access to create a Google Cloud project under that account. This setup takes about 10 minutes and only needs to be done once.

One-Time Setup — Google Cloud Console

1 Sign in to Google Cloud Console

Go to console.cloud.google.com and sign in with the **restaurant's Google Workspace account** (not your personal Gmail). Create a new project — name it "Rinata Agents" or similar.

2 Enable the Gmail API

In the left sidebar: **APIs & Services** → **Library**. Search for "Gmail API" and click **Enable**.

3 Create OAuth 2.0 Credentials

Go to **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth 2.0 Client ID**. Choose **Desktop app** as the application type. Name it anything. Click Create.

4 Download credentials.json

Click the download icon next to the credential you just created. Save the file as `credentials.json` and place it at:

```
C:\Users\shant\projects\Rinata\email_triage\credentials.json
```

5 Configure Your Suppliers

Open `email_triage/suppliers.json` and add the restaurant's actual supplier email domains. The file already has Sysco, US Foods, and Gordon Food Service as examples — update with whoever actually delivers to Rinata.

```
// Example entry — add your actual suppliers { "name": "Local Produce Co", "domains":  
["localproduceco.com"], "category": "Produce" }
```

6 Authenticate (browser opens once)

Run the auth script from your terminal. A browser window will open — sign in with the restaurant's Google account and grant permission. This creates `token.json` and you won't need to do this again.

```
python email_triage/gmail_auth.py # Expected output: "✓ Authenticated as: rinata@... "
```

Running the Pipeline

Full run (fetch new emails + categorize + build workbook)

```
python email_triage/run_email_triage.py
```

Fetch last 7 days only

```
python email_triage/run_email_triage.py --days 7
```

Re-categorize without fetching (useful after editing the prompt)

```
python email_triage/run_email_triage.py --skip-fetch
```

Smoke test — categorize only 5 emails

```
python email_triage/categorize_emails.py 5
```

Idempotency

Every script is safe to re-run. Emails already fetched or categorized are skipped automatically — you'll never pay twice to process the same message.

Recommended Cadence

Run `run_email_triage.py` every weekday morning before the manager arrives. Takes under 2 minutes. Open `Rinata_Email_Triage.xlsx` → Sheet 2 (Urgent Queue) first, then Sheet 3 (Invoices) for payment review.

Build 2 Hardening & Approval Automation

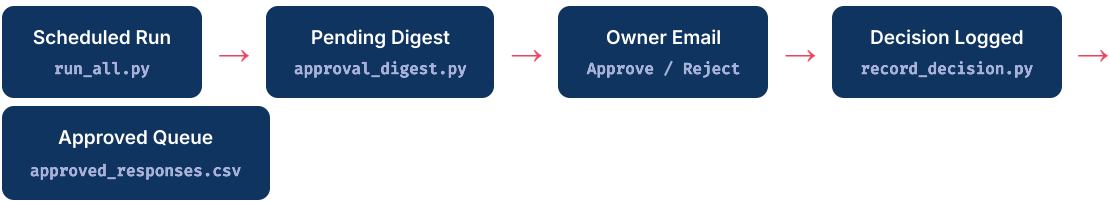
This addendum records work completed after Build 2's initial sign-off. The agent was production-hardened, gated behind an owner approval step, and wired for unattended scheduling. The architecture principles are unchanged — deterministic Python does the work, Claude is used only for the response wording, and every run stays auditable and cheap to operate.

What Was Added to Build 2

Enhancement	What it does	Why it matters
3★-and-below filter	Drafts replies only for reviews rated 3 stars or lower.	Ownership only responds to critical and negative feedback. Cuts the working set from 386 reviews to 32, drops API cost to near-zero, and focuses attention where it counts.
Pipeline orchestrator run_all.py	One command runs clean → categorize → workbook → responses → sheets in the required order, with sanity checks between steps.	Makes the suite safe to schedule unattended. Aborts if the scraper returns suspicious data instead of letting it flow downstream, and enforces step order so the response sheets are never overwritten.
Idempotent reruns	Skips reviews already approved or rejected; carries still-pending drafts forward unchanged.	Re-running never duplicates work, never rewrites a draft the owner is mid-review on, and never pays the Claude API twice — the same idempotency standard as Build 6.
Approval workflow	Every draft is held as "Pending Review" until the owner explicitly approves or rejects it.	No AI-written text reaches a public Google review without a human decision.

Approval Automation (n8n)

A scheduled n8n workflow drives the human-in-the-loop process end to end:



Docker-aware design

Because n8n runs inside Docker — isolated from the host's Python, files, and Chrome — the workflow uses a file-handoff design over a mounted volume rather than calling host commands. The host runs the pipeline on a schedule; n8n only sends the approval emails and writes decisions back into the shared output/ folder. n8n stays a thin orchestrator, and no Python or browser has to live in the container.

Honest limitation — no auto-posting

Build 1 scrapes Google reviews; it cannot post replies. "Approved" means a clean, owner-approved list (approved_responses.csv) the manager pastes into Google Business Profile. True auto-posting would require the Google Business Profile API and is deliberately out of scope — AI text is never published unattended.

Infrastructure & Tooling Decisions

Two infrastructure items were addressed alongside the Build 2 work. Both are recorded here with their rationale so the decisions are not revisited without cause.

GitHub MCP Server — Connected

The official GitHub MCP server was connected to Claude Code via Docker and registered at user scope, so it is available in every future session. This lets the assistant read pull requests and repository state directly during development work.

```
docker run -i --rm -e GITHUB_PERSONAL_ACCESS_TOKEN ghcr.io/github/github-mcp-server # Status: ✓ Connected –  
verified via `claude mcp list`
```

Local Models — Evaluated & Declined

Docker Model Runner was enabled and smoke-tested with a small local model. It works, but CPU-class local models are far weaker at instruction-following — directly at odds with this suite's governing principle: **accuracy over speed, because this is financial data** (most critical for Builds 3 & 4). The Claude API costs under a dollar for the entire review set.

Decision — the suite stays on the Claude API

Docker Model Runner remains installed but idle. Local inference will be reconsidered only if a hard data-residency requirement appears (financial data contractually cannot leave the machine) — and even then with a properly-sized GPU model, not a small CPU one.

Complete File Structure & Roadmap

Full Project File Structure

Rinata/

|

run_all.py

← Builds 1+2 orchestrator + sanity checks

|

.env

← ANTHROPIC_API_KEY (keep private)

|

scrapper/

|

scrape_reviews.py

← Build 1: Playwright scraper for Google reviews

|

categorizer/

|

clean_data.py

← Build 1: Data cleaning and normalization

|

categorize.py

← Build 1: Claude review categorization

|

build_excel.py

← Build 1: Analysis workbook (Sheets 1–4)

|

build_report.py

← Build 1: Management report

|

responder/

|

voice_guidelines.py

← Build 2: Brand voice source of truth

|

generate_responses.py

← Build 2: Claude response drafting

|

build_response_sheets.py

← Build 2: Adds Sheets 5–6 to workbook

|

record_decision.py

← Build 2: Approval workflow

|

approval_digest.py

← Build 2: Digest of pending approvals

|

email_triage/

← Build 6 (NEW)

|

suppliers.json

← Supplier domains + search keywords

|

gmail_auth.py

← One-time OAuth setup

|

fetch_emails.py

← Gmail API fetcher

|

categorize_emails.py

← Claude Haiku categorizer

|

build_triage_sheet.py

← Excel workbook builder

|

run_email_triage.py

← Pipeline orchestrator

|

prompts/

|

classify_prompt.txt

← Build 1: Review classification prompt

|

response_positive_v1.txt

← Build 2: Positive review response prompt

|

response_negative_v1.txt

← Build 2: Negative review response prompt

|

response_mixed_v1.txt

← Build 2: Mixed review response prompt

|

email_triage_v1.txt

← Build 6: Email categorization prompt (NEW)

|

docs/

← Build 2 (NEW): automation guides

|

n8n_automation.md

← n8n approval workflow build guide

|

n8n_workflow.json

← Importable n8n workflow scaffold

|

output/

← All generated files live here

|

raw_reviews.csv

← Build 1 output

|

clean_reviews.csv

← Build 1 output

|

categorized_reviews.csv

← Build 1 output

|

response_drafts.csv

← Build 2 output (pending, standard)

|

urgent_responses.csv

← Build 2 output (pending, urgent)

|

approved_responses.csv

← Build 2 output (owner-approved)

|

rejected_responses.csv

← Build 2 output (owner-rejected)

|

Rinata_Review_Analysis.xlsx

← Builds 1+2 deliverable

|

raw_emails.json

← Build 6 output

|

email_categories.csv

← Build 6 output

|

urgent_emails.csv

← Build 6 output

|

Rinata_Email_Triage.xlsx

← Build 6 deliverable

|

Deliverables/

← Final outputs for ownership review

Build Roadmap

Build 1	Feedback & Reviews Intelligence System		✓ Complete
Build 2	Review Response Agent		✓ Complete
Build 6	Supplier Email Triage Agent		Built — Setup Needed
Build 3	Weekly P&L Drafting Agent	Waiting on QuickBooks login + P&L targets sign-off	Blocked
Build 4	Vendor Invoice Reconciliation Agent	Waiting on vendor invoice PDFs + master price list	Blocked
Build 5	Menu Profitability Analyzer	Not yet scoped	Planned
Build 7	Scheduling Assistant Agent	Highest complexity — saved for last	Planned
Build 8	Reservation Pattern Analyzer	Not yet scoped	Planned

Next Actions to Unblock Builds 3 & 4

Build 3:

Request QuickBooks login from ownership + agree on weekly P&L targets (food cost %, labor %, prime cost %).

Build 4:

Gather the last 4–6 invoices from your top 5 vendors to build the master price list. Vendor invoice PDFs can be forwarded from

the restaurant email.